

Top K frequent elements

1. Brute force method

The screenshot shows a submission on NeetCode for the problem "Top K frequent elements". The submission is accepted, with a runtime of 28ms (beating 88.07%) and a memory usage of 8.0 MB (beating 78.95%). The code is a Python class `Solution` with a method `topKFrequent` that uses a dictionary to count the frequency of each number and then sorts the dictionary by value in descending order to return the top k elements.

```
1 class Solution:
2     def topKFrequent(self, nums: List[int], k: int) -> List[int]:
3         count = {}
4         for num in nums:
5             count[num] = count.get(num, 0) + 1
6         sorted_count = sorted(count.items(), key=lambda x: x[1], reverse=True)
7         return [num for num, _ in sorted_count[:k]]
8
```

The left sidebar shows the submission details: "Accepted 21 / 21 test cases", "Memory: 8 MB", "Time: 28ms", and "Submitted at: 06/01/2026 16:52". A bar chart shows the runtime distribution, with a peak at 26ms. The bottom right shows the "Run" and "Submit" buttons.

2. Optimal approach → hash map bucket sort

The screenshot shows a submission on NeetCode for the problem "Top K frequent elements". The submission is accepted, with a runtime of 28ms (beating 88.07%) and a memory usage of 8.0 MB (beating 78.95%). The code is a Python class `Solution` with a method `topKFrequent` that uses a dictionary to count the frequency of each number and then uses bucket sort to find the top k elements.

```
1 class Solution:
2     def topKFrequent(self, nums: List[int], k: int) -> List[int]:
3         count = defaultdict(int)
4         for num in nums:
5             count[num] += 1
6         freq = [[] for _ in range(len(nums)+1)]
7         for num, c in count.items():
8             freq[c].append(num)
9         res = []
10        for i in range(len(freq)-1, 0, -1):
11            for num in freq[i]:
12                res.append(num)
13            if len(res) == k:
14                return res
15
```

The left sidebar shows the submission details: "Accepted 21 / 21 test cases", "Memory: 8 MB", "Time: 28ms", and "Submitted at: 06/01/2026 17:16". A bar chart shows the runtime distribution, with a peak at 26ms. The bottom right shows the "Run" and "Submit" buttons.

Comparison:

General Overview		
	Brute Force	Optimized
Approach	Count + Sort	Count + Bucket Index
Time Complexity	$O(n \log n)$	$O(n)$
Space Complexity	$O(n)$	$O(n)$
Lines of Code	Less	More
Difficulty to Code	Easy	Medium
Chance of Bugs	Low	Medium
Best for Large Input	✗	✓

Step by Step		
Step	Brute Force	Optimized
Step 1	Count freq using HashMap	Count freq using HashMap
Step 2	Sort keys by frequency	Create bucket array size $n+1$
Step 3	Slice first k elements	Read right to left, collect k
Bottleneck	Sorting $\rightarrow O(n \log n)$	No bottleneck $\rightarrow O(n)$

Key Concepts Used		
	Brute Force	Optimized
Data Structure	HashMap	HashMap + Array
Algorithm	Sorting	Bucket Sort
Key Insight	Sort by value of HashMap	Frequency as array index
Built-in Used	sorted()	None